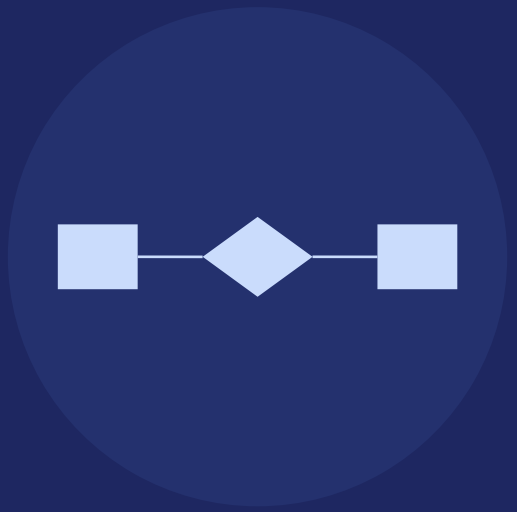
MODELAGEM DE DADOS · AULA

Consultas em SQL

Do SELECT básico às subconsultas

Catálogo de Filmes · SQLite · DB Browser for SQLite



DO MAIS SIMPLES AO MAIS COMPLETO

Roteiro da Aula

Cada degrau usa o que foi visto no anterior.

1

Consultas básicas

- SELECT e DISTINCT
- WHERE, AND/OR, BETWEEN, IN, LIKE
- NULL e COALESCE
- ORDER BY e LIMIT
- Funções e colunas calculadas

2

Agregação

- COUNT, SUM, AVG, MIN, MAX
- GROUP BY
- HAVING
- Ordem de execução

3

Junções

- Produto cartesiano e WHERE
- INNER JOIN
- Várias tabelas e N:N
- JOIN + GROUP BY
- INNER × LEFT JOIN
- LEFT JOIN e IS NULL

4

Subconsultas e mais

- Subconsultas, IN, EXISTS
- INTERSECT e EXCEPT
- CASE WHEN
- WITH (CTE)

ANTES DE COMEÇAR

Preparando o Ambiente

1. Abra o DB Browser for SQLite e crie um banco novo (por exemplo, filmes.db)
2. Na aba Executar SQL, abra e execute catalogo_filmes.sql (cria as tabelas)
3. Abra e execute catalogo_filmes_dados.sql (insere os dados de exemplo)
4. Salve as alterações (Write Changes, Ctrl+S) — sem isso a carga não é gravada
5. Confira a carga com a consulta ao lado: devem aparecer 24 filmes

CONFERINDO A CARGA

```sql
SELECT COUNT(*) AS filmes FROM filme;
```

filmes
24

Resultado: 1 linha

Se algo der errado, apague o banco e recomece pelo passo 1 — os scripts podem ser rodados quantas vezes forem necessárias.

CATÁLOGO DE FILMES

O Banco que Vamos Consultar

Catálogo		Elenco e direção		Usuários	
`filme`	24 linhas	`profissional`	83 linhas	`usuario`	11 linhas
`estudio`	14 linhas	`atuacao`	73 linhas	`favorito`	39 linhas
`classificacao`	6 linhas	`direcao`	25 linhas	`comentario`	55 linhas
`pais`	11 linhas	`filme_genero`	55 linhas		
`genero`	13 linhas				

As relações entre elas (chaves estrangeiras) são o que permite as junções do Bloco 3.

A ORDEM EM QUE ESCREVEMOS

Anatomia do SELECT

`SELECT`	quais colunas ou expressões mostrar	obrigatório
`FROM`	de qual(is) tabela(s) vêm os dados	obrigatório*
`WHERE`	filtra as linhas antes de agrupar	opcional
`GROUP BY`	agrupa linhas que têm o mesmo valor	opcional
`HAVING`	filtra os grupos (depois do GROUP BY)	opcional
`ORDER BY`	ordena o resultado	opcional
`LIMIT`	limita a quantidade de linhas	opcional

** O FROM só pode faltar em consultas sem tabela (como SELECT 1 + 1). Nesta aula, sempre o usamos. A ordem de escrita é fixa.*

BLOCO 1 · CONSULTAS BÁSICAS

SELECT *: todas as colunas

Quais são as classificações indicativas cadastradas?

```
SELECT * FROM classificacao;
```

RESULTADO · 6 linhas

cod_class	descricao
1	Livre
2	10
3	12
4	14
5	16
6	18

- O asterisco devolve todas as colunas, na ordem em que foram criadas
- Ótimo para explorar uma tabela; em consultas definitivas, prefira listar as colunas

BLOCO 1 · CONSULTAS BÁSICAS

Escolhendo Colunas e Apelidos

Título, ano e duração de todos os filmes, com nomes mais amigáveis

```
SELECT titulo,
       ano_lancamento AS ano,
       duracao AS minutos
FROM filme;
```

RESULTADO · 24 linhas (mostrando 7)

titulo	ano	minutos
Cidade de Deus	2002	130
Central do Brasil	1998	110
Parasita	2019	132
A Viagem de Chihiro	2001	125
Pulp Fiction: Tempo de Violência	1994	154
Kill Bill: Volume 1	2003	111
O Poderoso Chefão	1972	175

- Liste só as colunas necessárias, separadas por vírgula
- AS dá um apelido à coluna no resultado (a tabela não muda)

BLOCO 1 · CONSULTAS BÁSICAS

DISTINCT: Removendo Repetições

Quais notas já foram dadas nos comentários (sem repetir)?

```
SELECT DISTINCT nota
FROM comentario;
```

RESULTADO · 4 linhas

nota
5
4
3
2

- Os 55 comentários viram apenas 4 valores diferentes
- Com várias colunas, DISTINCT vale para a combinação delas

BLOCO 1 · CONSULTAS BÁSICAS

WHERE: Filtrando Linhas

Quais filmes foram lançados a partir de 2010?

```
SELECT titulo, ano_lancamento
FROM filme
WHERE ano_lancamento >= 2010;
```

RESULTADO · 4 linhas

titulo	ano_lancamento
Parasita	2019
Argo	2012
Interestelar	2014
A Origem	2010

- WHERE vem depois do FROM e decide quais linhas entram no resultado
- Textos vão entre aspas simples: WHERE titulo = 'Matrix'

`=` `<>` `>` `<` `>=` `<=` *operadores de comparação*

BLOCO 1 · CONSULTAS BÁSICAS

AND, OR e NOT

Filmes de 2000 em diante com mais de 150 minutos

```
SELECT titulo, ano_lancamento, duracao
FROM filme
WHERE ano_lancamento >= 2000
  AND duracao > 150;
```

RESULTADO · 4 linhas

titulo	ano_lancamento	duracao
Interestelar	2014	169
Batman: O Cavaleiro das Trevas	2008	152
Avatar	2009	162
O Senhor dos Anéis: A Sociedade do Anel	2001	178

- AND: as duas condições precisam ser verdadeiras. OR: basta uma. NOT: nega a condição
- O AND é avaliado antes do OR — use parênteses quando misturar os dois

BLOCO 1 · CONSULTAS BÁSICAS

BETWEEN: Intervalos

Filmes lançados na década de 1990

```
SELECT titulo, ano_lancamento
FROM filme
WHERE ano_lancamento BETWEEN 1990 AND 1999;
```

RESULTADO · 7 linhas

titulo	ano_lancamento
Central do Brasil	1998
Pulp Fiction: Tempo de Violência	1994
Os Imperdoáveis	1992
Matrix	1999
Titanic	1997
Toy Story	1995
O Silêncio dos Inocentes	1991

- BETWEEN inclui os dois extremos (1990 e 1999 entram)
- É o mesmo que ano_lancamento >= 1990 AND ano_lancamento <= 1999

BLOCO 1 · CONSULTAS BÁSICAS

IN: Lista de Valores

Filmes lançados em 2001 ou em 2003

```
SELECT titulo, ano_lancamento
FROM filme
WHERE ano_lancamento IN (2001, 2003);
```

RESULTADO · 5 linhas

titulo	ano_lancamento
A Viagem de Chihiro	2001
Kill Bill: Volume 1	2003
O Senhor dos Anéis: A Sociedade do Anel	2001
Procurando Nemo	2003
O Fabuloso Destino de Amélie Poulain	2001

- Equivale a vários OR: ano = 2001 OR ano = 2003
- NOT IN faz o contrário: exclui os valores da lista

BLOCO 1 · CONSULTAS BÁSICAS

LIKE: Busca por Padrão

Títulos que começam com "O " (% = qualquer sequência)

```
SELECT titulo
FROM filme
WHERE titulo LIKE 'O %';
```

RESULTADO · 5 linhas (mostrando 4)

titulo
O Poderoso Chefão
O Senhor dos Anéis: A Sociedade do Anel
O Iluminado
O Silêncio dos Inocentes

Título com uma letra qualquer no meio (_ = exatamente uma letra)

```
SELECT titulo
FROM filme
WHERE titulo LIKE 'M_trix';
```

RESULTADO · 1 linha

titulo
Matrix

% representa qualquer sequência de letras e _ representa exatamente uma. No SQLite, o LIKE não diferencia maiúsculas de minúsculas (letras ASCII).

BLOCO 1 · CONSULTAS BÁSICAS

NULL: Valor Ausente

Quais filmes não têm sinopse cadastrada?

```
SELECT titulo
FROM filme
WHERE sinopse IS NULL;
```

RESULTADO · 3 linhas

titulo
Toy Story
Procurando Nemo
Psicose

Trocar o texto ausente por um valor padrão

```
SELECT cod_coment, nota,
       COALESCE(texto, '(sem texto)') AS texto
FROM comentario
WHERE texto IS NULL;
```

RESULTADO · 3 linhas

cod_coment	nota	texto
7	5	(sem texto)
24	3	(sem texto)
51	2	(sem texto)

Teste NULL com IS NULL / IS NOT NULL: escrever = NULL nunca retorna linhas. COALESCE devolve o primeiro valor que não for NULL.

BLOCO 1 · CONSULTAS BÁSICAS

ORDER BY e LIMIT

Os 5 filmes mais longos

```
SELECT titulo, duracao
FROM filme
ORDER BY duracao DESC
LIMIT 5;
```

RESULTADO · 5 linhas

titulo	duracao
Titanic	194
O Senhor dos Anéis: A Sociedade do Anel	178
O Poderoso Chefão	175
Interestelar	169
Avatar	162

- ASC (padrão) ordena do menor para o maior; DESC, do maior para o menor
- Dá para ordenar por várias colunas: ORDER BY ano_lancamento DESC, titulo
- LIMIT n devolve apenas as n primeiras linhas, depois da ordenação

BLOCO 1 · CONSULTAS BÁSICAS

Colunas Calculadas e Funções

Tamanho do título e duração em horas, para filmes com mais de 170 min

```
SELECT titulo,
       LENGTH(titulo) AS caracteres,
       ROUND(duracao / 60.0, 1) AS horas
FROM filme
WHERE duracao > 170;
```

RESULTADO · 3 linhas

titulo	caracteres	horas
O Poderoso Chefão	17	2.9
Titanic	7	3.2
O Senhor dos Anéis: A Sociedade do Anel	39	3.0

- Expressões e funções criam colunas calculadas na hora, sem alterar a tabela
- Algumas funções: ROUND, LENGTH, UPPER, LOWER, SUBSTR, COALESCE
- UPPER e LOWER do SQLite só convertem letras ASCII: acentos ficam como estão

BLOCO 2 · AGREGAÇÃO

Funções de Agregação

Um resumo geral dos filmes cadastrados

```
SELECT COUNT(*) AS total,
       MIN(ano_lancamento) AS mais_antigo,
       MAX(ano_lancamento) AS mais_novo,
       ROUND(AVG(duracao), 1) AS duracao_media
FROM filme;
```

RESULTADO · 1 linha

total	mais_antigo	mais_novo	duracao_media
24	1941	2019	135.5

COUNT(*) conta linhas; COUNT(coluna) ignora os NULL

```
SELECT COUNT(*) AS filmes,
       COUNT(sinopse) AS com_sinopse
FROM filme;
```

RESULTADO · 1 linha

filmes	com_sinopse
24	21

COUNT, SUM, AVG, MIN e MAX resumem várias linhas em um valor. Sem GROUP BY, o resultado é uma única linha; os NULL são ignorados (exceto em COUNT(*)).

BLOCO 2 · AGREGAÇÃO

GROUP BY: Um Resultado por Grupo

Quantos comentários cada filme recebeu e qual a nota média?

```
SELECT cod_filme,
       COUNT(*) AS comentarios,
       ROUND(AVG(nota), 2) AS media
FROM comentario
GROUP BY cod_filme;
```

RESULTADO · 21 linhas (mostrando 7)

cod_filme	comentarios	media
1	3	4.67
3	4	4.75
4	2	5.0
5	2	4.5
6	2	3.5
7	3	4.67
8	1	4.0

- Cada linha do resultado representa um grupo (aqui, um filme)
- Toda coluna do SELECT que não é agregada precisa estar no GROUP BY
- Só vemos o código do filme: no Bloco 3, o JOIN vai trazer o título

BLOCO 2 · AGREGAÇÃO

HAVING: Filtrando Grupos

Apenas os filmes com pelo menos 4 comentários

```sql
SELECT cod_filme,
       COUNT(*) AS comentarios,
       ROUND(AVG(nota), 2) AS media
FROM comentario
GROUP BY cod_filme
HAVING COUNT(*) >= 4;
```

RESULTADO · 4 linhas

cod_filme	comentarios	media
3	4	4.75
12	6	4.33
13	4	4.75
15	4	4.75

- WHERE filtra linhas antes de agrupar; HAVING filtra grupos depois
- HAVING pode usar agregações (COUNT, AVG...); o WHERE não pode

BLOCO 2 · AGREGAÇÃO

Ordem de Execução

Escrevemos o SELECT em uma ordem, mas o banco o executa em outra:



- Por isso o WHERE não pode usar COUNT ou AVG: os grupos ainda não existem. Para filtrar grupos, existe o HAVING.
- No SQL padrão, o WHERE não enxerga apelidos criados no SELECT (o SQLite tolera, mas outros SGBDs não).
- O ORDER BY roda depois do SELECT, então pode usar apelidos, como ORDER BY media.

BLOCO 3 · JUNÇÕES (JOIN)

Por que JOIN?

Os dados ficam separados em tabelas, ligados por chaves. O JOIN junta as linhas que se correspondem.

`filme`

cod_filme	titulo	cod_estudio
4	A Viagem de Chihiro	**7**
12	Matrix	**1**
19	Toy Story	**6**

`cod_estudio`

FK ↔ PK

`estudio`

cod_estudio	nome
1	Warner Bros.
6	Pixar
7	Studio Ghibli

```
JOIN  filme.cod_estudio = estudio.cod_estudio
```

titulo	estudio
A Viagem de Chihiro	Studio Ghibli
Matrix	Warner Bros.
Toy Story	Pixar

O resultado traz o título (de filme) e o nome do estúdio (de estudio) na mesma linha.

BLOCO 3 · JUNÇÕES (JOIN)

Duas Tabelas no FROM: Produto Cartesiano

Duas tabelas no FROM, separadas por vírgula e sem nenhuma condição

```
SELECT f.titulo, e.nome AS estudio
FROM filme f, estudio e
ORDER BY f.titulo, e.nome;
```

RESULTADO · 336 linhas (mostrando 7)

titulo	estudio
A Origem	20th Century Fox
A Origem	Barunson E&A
A Origem	Miramax
A Origem	New Line Cinema
A Origem	O2 Filmes
A Origem	Orion Pictures
A Origem	Paramount Pictures

- 24 filmes × 14 estúdios = 336 linhas: cada filme aparece ao lado de todos os estúdios
- Só 24 dessas combinações são verdadeiras: a de cada filme com o seu estúdio
- Falta dizer ao banco quais combinações interessam: é a condição de junção

BLOCO 3 · JUNÇÕES (JOIN)

WHERE como Junção

Agora com a condição no WHERE: chave estrangeira = chave primária

```
SELECT f.titulo, e.nome AS estudio
FROM filme f, estudio e
WHERE e.cod_estudio = f.cod_estudio
ORDER BY f.titulo;
```

RESULTADO · 24 linhas (mostrando 7)

titulo	estudio
A Origem	Warner Bros.
A Viagem de Chihiro	Studio Ghibli
Argo	Warner Bros.
Avatar	20th Century Fox
Batman: O Cavaleiro das Trevas	Warner Bros.
Central do Brasil	VideoFilmes
Cidade de Deus	O2 Filmes

- O WHERE mantém só as linhas em que a chave estrangeira (f.cod_estudio) é igual à chave primária (e.cod_estudio)
- 336 linhas viram 24: é uma junção feita “na mão”
- O resultado é idêntico ao do INNER JOIN, que veremos a seguir

BLOCO 3 · JUNÇÕES (JOIN)

INNER JOIN: a Sintaxe Explícita

Cada filme com o nome do seu estúdio

```
SELECT f.titulo, e.nome AS estudio
FROM filme f
INNER JOIN estudio e
        ON e.cod_estudio = f.cod_estudio
ORDER BY f.titulo;
```

RESULTADO · 24 linhas (mostrando 4)

titulo	estudio
A Origem	Warner Bros.
A Viagem de Chihiro	Studio Ghibli
Argo	Warner Bros.
Avatar	20th Century Fox

Apenas os filmes da Pixar (JOIN + WHERE)

```
SELECT f.titulo, f.ano_lancamento
FROM filme f
JOIN estudio e ON e.cod_estudio = f.cod_estudio
WHERE e.nome = 'Pixar';
```

RESULTADO · 2 linhas

titulo	ano_lancamento
Toy Story	1995
Procurando Nemo	2003

Mesmo resultado do WHERE da página anterior, agora com a condição no ON. Os apelidos f e e abreviam os nomes das tabelas.

BLOCO 3 · JUNÇÕES (JOIN)

Por que Preferir o JOIN?

1 Cada coisa no seu lugar

No JOIN, a ligação entre as tabelas fica no ON e os filtros ficam no WHERE. Na vírgula, os dois se misturam no mesmo WHERE, e a consulta fica mais difícil de ler.

2 A condição fica visível

O ON deixa a ligação explícita. No padrão SQL ele é obrigatório, mas o SQLite aceita JOIN sem ON e devolve as 336 linhas do produto cartesiano. Por isso, sempre confira o ON.

3 O LEFT JOIN só existe assim

Com vírgula e WHERE, a Universal Pictures nunca aparece (0 linhas), porque não tem filmes. Para mantê-la no resultado, é preciso o LEFT JOIN.

Ressalva: a vírgula é a sintaxe mais antiga. Serve como ponto de partida para entender o JOIN; no dia a dia, use JOIN … ON.

BLOCO 3 · JUNÇÕES (JOIN)

JOIN com Várias Tabelas

Quem atua em Matrix e qual personagem? Primeiro, com vírgula e WHERE

```
SELECT p.nome AS ator, a.personagem
FROM filme f, atuacao a, profissional p
WHERE a.cod_filme = f.cod_filme
  AND p.cod_prof = a.cod_prof
  AND f.titulo = 'Matrix';
```

RESULTADO · 3 linhas

ator	personagem
Keanu Reeves	Neo
Laurence Fishburne	Morpheus
Carrie-Anne Moss	Trinity

O mesmo resultado, agora com JOIN e ON

```
SELECT p.nome AS ator, a.personagem
FROM filme f
JOIN atuacao a ON a.cod_filme = f.cod_filme
JOIN profissional p ON p.cod_prof = a.cod_prof
WHERE f.titulo = 'Matrix';
```

RESULTADO · 3 linhas

ator	personagem
Keanu Reeves	Neo
Laurence Fishburne	Morpheus
Carrie-Anne Moss	Trinity

Cada JOIN acrescenta uma tabela (filme → atuacao → profissional), e atuacao é a associativa do N:N. Colunas com o mesmo nome pedem o prefixo do apelido (f.cod_filme).

BLOCO 3 · JUNÇÕES (JOIN)

JOIN em Relacionamento N:N

Quais filmes são de Ficção Científica?

```
SELECT f.titulo
FROM filme f
JOIN filme_genero fg ON fg.cod_filme = f.cod_filme
JOIN genero g ON g.cod_genero = fg.cod_genero
WHERE g.nome = 'Ficção Científica'
ORDER BY f.titulo;
```

RESULTADO · 4 linhas

titulo
A Origem
Avatar
Interestelar
Matrix

- Duas junções passam pela tabela associativa filme_genero
- O WHERE filtra pelo nome do gênero, que vem de outra tabela

BLOCO 3 · JUNÇÕES (JOIN)

JOIN + GROUP BY

Os 5 filmes com melhor nota média (agora com o título)

```
SELECT f.titulo,
       COUNT(*) AS comentarios,
       ROUND(AVG(c.nota), 2) AS media
FROM filme f
JOIN comentario c ON c.cod_filme = f.cod_filme
GROUP BY f.cod_filme, f.titulo
ORDER BY media DESC, comentarios DESC, f.titulo
LIMIT 5;
```

RESULTADO · 5 linhas

titulo	comentarios	media
A Viagem de Chihiro	2	5.0
Toy Story	1	5.0
Batman: O Cavaleiro das Trevas	4	4.75
Interestelar	4	4.75
Parasita	4	4.75

- Agora com o título em vez do código: o JOIN resolve o problema do Bloco 2
- O GROUP BY lista as colunas não agregadas (código e título)
- No ORDER BY, vários critérios desempatam: média, depois comentários, depois título

BLOCO 3 · JUNÇÕES (JOIN)

INNER JOIN × LEFT JOIN

INNER JOIN: só aparece quem tem par nas duas tabelas

```sql
SELECT e.nome AS estudio, f.titulo
FROM estudio e
INNER JOIN filme f ON f.cod_estudio = e.cod_estudio
WHERE e.nome IN ('Pixar', 'Universal Pictures')
ORDER BY e.nome, f.titulo;
```

RESULTADO · 2 linhas

estudio	titulo
Pixar	Procurando Nemo
Pixar	Toy Story

LEFT JOIN: todos os estúdios, mesmo sem filmes

```sql
SELECT e.nome AS estudio, f.titulo
FROM estudio e
LEFT JOIN filme f ON f.cod_estudio = e.cod_estudio
WHERE e.nome IN ('Pixar', 'Universal Pictures')
ORDER BY e.nome, f.titulo;
```

RESULTADO · 3 linhas

estudio	titulo
Pixar	Procurando Nemo
Pixar	Toy Story
Universal Pictures	*NULL*

O LEFT JOIN mantém todas as linhas da tabela da esquerda; quando não há par, as colunas da direita vêm como NULL.

BLOCO 3 · JUNÇÕES (JOIN)

INNER × LEFT JOIN: Quando Usar Cada Um

INNER JOIN

- Devolve só as linhas que têm par nas duas tabelas
- Quem não tem par some do resultado

Use quando só interessa o que tem relação, como cada filme com o seu estúdio

```
estudio INNER JOIN filme  →  24 linhas
```

LEFT JOIN

- Devolve todas as linhas da tabela da esquerda
- Sem par, as colunas da direita vêm como NULL

Use quando não se pode perder as linhas da esquerda (todos os estúdios, mesmo sem filmes) ou para achar quem não tem par, com IS NULL

```
estudio LEFT JOIN filme  →  25 linhas (a Universal entra com NULL)
```

A ordem das tabelas importa no LEFT JOIN: estudio LEFT JOIN filme preserva os estúdios (25 linhas); filme LEFT JOIN estudio preservaria os filmes (24 linhas, todos têm estúdio).

BLOCO 3 · JUNÇÕES (JOIN)

LEFT JOIN + COUNT: Contando Zeros

Quantos filmes cada estúdio produziu, incluindo quem tem zero?

```
SELECT e.nome AS estudio,
       COUNT(f.cod_filme) AS filmes
FROM estudio e
LEFT JOIN filme f ON f.cod_estudio = e.cod_estudio
GROUP BY e.cod_estudio, e.nome
ORDER BY filmes, estudio;
```

RESULTADO · 14 linhas (mostrando 6)

estudio	filmes
Universal Pictures	0
Barunson E&A	1
New Line Cinema	1
O2 Filmes	1
Orion Pictures	1
RKO Pictures	1

- COUNT(f.cod_filme) conta só filmes que existem, então a Universal fica com 0
- COUNT(*) contaria 1 para ela, porque a linha com NULL também é uma linha

BLOCO 3 · JUNÇÕES (JOIN)

LEFT JOIN + IS NULL: Quem Não Tem Par

Filmes que ainda não receberam nenhum comentário

```
SELECT f.titulo
FROM filme f
LEFT JOIN comentario c ON c.cod_filme = f.cod_filme
WHERE c.cod_coment IS NULL;
```

RESULTADO · 3 linhas

titulo
Central do Brasil
Cidadão Kane
Psicose

Usuários que não têm nenhum filme favorito

```
SELECT u.nome
FROM usuario u
LEFT JOIN favorito fa ON fa.cod_usuario = u.cod_usuario
WHERE fa.cod_filme IS NULL;
```

RESULTADO · 1 linha

nome
João Pedro Alves

Padrão útil: faça o LEFT JOIN e filtre pelo NULL da tabela da direita para encontrar o que não tem correspondência.

BLOCO 4 · SUBCONSULTAS E MAIS

Subconsulta: Um Valor Calculado

Filmes mais longos que a duração média

```
SELECT titulo, duracao
FROM filme
WHERE duracao > (SELECT AVG(duracao)
                 FROM filme)
ORDER BY duracao DESC;
```

RESULTADO · 10 linhas (mostrando 6)

titulo	duracao
Titanic	194
O Senhor dos Anéis: A Sociedade do Anel	178
O Poderoso Chefão	175
Interestelar	169
Avatar	162
Pulp Fiction: Tempo de Violência	154

- A subconsulta, entre parênteses, roda primeiro e devolve um único valor: 135.54 (a duração média)
- É como escrever duracao > 135.54 mas sempre atualizado com os dados



BLOCO 4 · SUBCONSULTAS E MAIS

Subconsulta com IN

Quem atua e também dirige?

```
SELECT nome
FROM profissional
WHERE cod_prof IN (SELECT cod_prof FROM atuacao)
  AND cod_prof IN (SELECT cod_prof FROM direcao)
ORDER BY nome;
```

RESULTADO · 4 linhas

nome
Ben Affleck
Clint Eastwood
Orson Welles
Quentin Tarantino

- A subconsulta devolve uma lista de códigos; o IN verifica se o código está nela
- Dois IN ligados por AND: quem aparece nas duas tabelas (atua e dirige)

BLOCO 4 · SUBCONSULTAS E MAIS

EXISTS e NOT EXISTS

Usuários que já comentaram algum filme (EXISTS)

```
SELECT u.nome
FROM usuario u
WHERE EXISTS (SELECT 1
              FROM comentario c
              WHERE c.cod_usuario = u.cod_usuario);
```

RESULTADO · 9 linhas (mostrando 3)

nome
Ana Souza
Bruno Lima
Carla Mendes

Usuários que nunca comentaram (NOT EXISTS)

```
SELECT u.nome
FROM usuario u
WHERE NOT EXISTS (SELECT 1
                  FROM comentario c
                  WHERE c.cod_usuario = u.cod_usuario);
```

RESULTADO · 2 linhas

nome
João Pedro Alves
Karina Torres

A subconsulta usa u.cod_usuario, da consulta externa (correlacionada). Prefira NOT EXISTS a NOT IN: se a subconsulta devolver algum NULL, o NOT IN não retorna nada.

BLOCO 4 · SUBCONSULTAS E MAIS

INTERSECT e EXCEPT

Profissionais que atuam e dirigem (INTERSECT)

```
SELECT cod_prof FROM atuacao
INTERSECT
SELECT cod_prof FROM direcao;
```

RESULTADO · 4 linhas

cod_prof
5
7
8
9

Profissionais que dirigem, mas nunca atuaram (EXCEPT)

```
SELECT cod_prof FROM direcao
EXCEPT
SELECT cod_prof FROM atuacao;
```

RESULTADO · 16 linhas (mostrando 4)

cod_prof
1
2
3
4

UNION junta resultados sem repetir (UNION ALL mantém as repetições). Nesses operadores, as consultas precisam ter o mesmo número de colunas, com tipos compatíveis.

BLOCO 4 · SUBCONSULTAS E MAIS

CASE WHEN: Lógica Condicional

Classificar cada filme pela duração

```
SELECT titulo, duracao,
       CASE WHEN duracao < 100 THEN 'curto'
            WHEN duracao <= 150 THEN 'médio'
            ELSE 'longo'
       END AS tamanho
FROM filme
ORDER BY duracao
LIMIT 4;
```

RESULTADO · 4 linhas

titulo	duracao	tamanho
Toy Story	81	curto
Procurando Nemo	100	médio
Psicose	109	médio
Central do Brasil	110	médio

- Funciona como um if / else dentro do SELECT: os WHEN são testados em ordem
- O primeiro que for verdadeiro define o valor; o ELSE cobre o restante
- Também pode ser usado em ORDER BY e GROUP BY

BLOCO 4 · SUBCONSULTAS E MAIS

WITH (CTE): Consulta com Nome

Filmes com nota média de 4,5 ou mais (WITH)

```sql
WITH media_filme AS (
  SELECT cod_filme,
         AVG(nota) AS media
  FROM comentario
  GROUP BY cod_filme
)
SELECT f.titulo,
       ROUND(m.media, 2) AS media
FROM filme f
JOIN media_filme m ON m.cod_filme = f.cod_filme
WHERE m.media >= 4.5
ORDER BY m.media DESC, f.titulo;
```

RESULTADO · 10 linhas (mostrando 6)

titulo	media
A Viagem de Chihiro	5.0
Toy Story	5.0
Batman: O Cavaleiro das Trevas	4.75
Interestelar	4.75
Parasita	4.75
Cidade de Deus	4.67

- A CTE é uma “tabela temporária” que existe só durante a consulta
- Calcula primeiro (média por filme) e junta depois, deixando o SQL mais legível

BLOCO 4 · SUBCONSULTAS E MAIS

Juntando Tudo

Gêneros com a melhor nota média (mínimo de 3 comentários)

```sql
SELECT g.nome AS genero,
       COUNT(*) AS comentarios,
       ROUND(AVG(c.nota), 2) AS media
FROM genero g
JOIN filme_genero fg ON fg.cod_genero = g.cod_genero
JOIN comentario c ON c.cod_filme = fg.cod_filme
GROUP BY g.cod_genero, g.nome
HAVING COUNT(*) >= 3
ORDER BY media DESC;
```

RESULTADO · 11 linhas (mostrando 6)

genero	comentarios	media
Fantasia	5	4.8
Animação	4	4.75
Comédia	8	4.5
Crime	16	4.5
Drama	27	4.37
Aventura	14	4.36

1 FROM + JOIN

une genero, filme_genero e comentario

2 GROUP BY

um grupo para cada gênero

3 HAVING

mantém só gêneros com 3 ou mais comentários

4 SELECT

calcula COUNT e AVG de cada grupo

5 ORDER BY

ordena pela média, da maior para a menor

PARA GUARDAR

Resumo: a Cola da Aula

Recurso	Para que serve
`SELECT / FROM`	escolhem as colunas e a tabela
`WHERE`	filtra linhas: = <> > < BETWEEN IN LIKE IS NULL AND OR NOT
`ORDER BY / LIMIT`	ordenam o resultado e limitam a quantidade de linhas
`COUNT, SUM, AVG, MIN, MAX`	agregam várias linhas em um valor
`GROUP BY / HAVING`	agrupam linhas e filtram os grupos
`INNER JOIN`	junta tabelas e mantém só as linhas que têm par
`LEFT JOIN`	mantém todas as linhas da esquerda (sem par, a direita vem NULL)
`Subconsulta (IN, EXISTS)`	uma consulta dentro da outra, devolvendo um valor ou uma lista
`INTERSECT / EXCEPT / UNION`	operações de conjunto entre resultados
`CASE WHEN / COALESCE`	lógica condicional e tratamento de NULL
`WITH (CTE)`	dá um nome a uma consulta intermediária

EXERCÍCIOS

Hora de Praticar

Básico

1. Liste título e ano dos filmes lançados antes de 1980, do mais antigo ao mais recente.

2. Quais estúdios não têm o ano de fundação informado?

Agregação

3. Quantos comentários cada nota (de 1 a 5) recebeu?

4. Em quais anos foram lançados mais de um filme? Mostre o ano e a quantidade.

JOIN

5. Liste título e estúdio dos filmes da Paramount Pictures.

6. Liste o elenco (ator e personagem) de Titanic.

Avançado

7. Quais filmes foram favoritados por pelo menos 3 usuários? Mostre título e quantidade.

8. Quantos filmes cada país produziu, incluindo os países sem nenhum filme?

Dica: confira cada resultado pensando se a quantidade de linhas faz sentido. O gabarito está nas notas do apresentador.



O que vocês aprenderam hoje

- Filtrar, ordenar e limitar linhas com WHERE, ORDER BY e LIMIT
- Resumir dados com funções de agregação, GROUP BY e HAVING
- Juntar tabelas com INNER JOIN e LEFT JOIN, inclusive em relacionamentos N:N
- Fazer perguntas mais elaboradas com subconsultas, CASE e WITH

Da modelagem conceitual ao SELECT: o ciclo completo do banco de dados.